CampusEventHub

Technical Documentation

Full-Stack MERN Web Application

Inter-College Event Management Platform

Technology Stack: MongoDB | Express.js | React.js | Node.js

1. Project Name

CampusEventHub

2. Description

CampusEventHub is a full-stack MERN (MongoDB, Express.js, React.js, Node.js) web application for centralized, real-time inter-college event management. It solves the problem of fragmented event information and manual registration by providing a unified, role-based platform for students, college admins, and superadmins.

The frontend (React.js) uses modular components, role-based routing, and dynamic dashboards for event discovery, registration, and management. The backend (Node.js/Express.js) exposes RESTful APIs for authentication, event CRUD, registration, notifications, and admin tools. MongoDB stores users, events, registrations, feedback, and notifications in separate collections for scalability and maintainability.

User roles are strictly enforced across the platform:

- Student: Discover, register, and track events; manage personal profile.
- College Admin: Create and manage events, approve or reject registrations, view analytics, manage profile.
- Superadmin: Approve or reject colleges and admins, view system logs, manage platform-wide analytics.

The platform is built with real-world full-stack practices including modular MVC architecture, secure authentication, robust error handling, and clear separation of frontend and backend responsibilities.

3. Purpose

The primary objectives of CampusEventHub are:

- Optimize event discovery and participation for students across colleges.
- Empower college admins with robust, real-time event management capabilities.
- Streamline approval, registration, and feedback workflows for all stakeholders.
- Ensure secure, scalable, and transparent campus activity management.

4. Technology Stack

Frontend: React.js — modular, role-based, dynamic UI components.

Backend: Node.js + Express.js — RESTful APIs with modular MVC architecture.

Database: MongoDB — Collections: users, events, registrations, feedback, notifications.

Authentication: JWT (stateless tokens), bcrypt (password hashing).

Email Service: SMTP — OTP delivery and event notifications.

5. Prerequisites

Before running the application, ensure the following are installed and configured:

- Node.js (v20 or higher recommended)
- npm (v9 or higher)
- MongoDB (local instance or MongoDB Atlas cloud database)
- SMTP email setup (Gmail, Outlook, or custom SMTP provider)
- IDE such as VS Code or any similar editor

6. Installation

Clone the repository and install all dependencies using the following commands:

```
git clone
https://github.com/yourusername/CampusEventHub_Team4.git
cd CampusEventHub_Team4
npm install
```

7. Environment Variables

The application requires environment variables configured in a .env file at the root of the backend directory. The table below lists all required variables:

Variable Name	Description	Example Value
USER_DB_URL	MongoDB URI for user data	mongodb://localhost:27017/users
EVENT_DB_URL	MongoDB URI for event data	mongodb://localhost:27017/events
NOTIFICATION_DB_URL	MongoDB URI for notifications	mongodb://localhost:27017/notify
APPLICATION_DB_URL	MongoDB URI for applications	mongodb://localhost:27017/app

Variable Name	Description	Example Value
JWT_SECRET	JWT signing key	mySuperSecretKey123
SMTP_HOST	SMTP server host	smtp.gmail.com
SMTP_PORT	SMTP server port	587
SMTP_USER	SMTP login email	admin@campuseventhub.com
SMTP_PASS	SMTP password	yourpassword

Note: Each database URL represents a separate collection or logical separation in MongoDB to maintain modular data handling. This improves scalability, maintainability, and avoids data coupling between users, events, notifications, and applications. Never commit the .env file to Git — add it to .gitignore to prevent credential leaks.

8. Running the Application

Backend

```
cd backend
nodemon server.js
```

The backend server runs on port 5000 by default.

Frontend

```
cd frontend
npm start
```

The frontend development server runs on port 3000 by default.

Accessing the Application

- Frontend: http://localhost:3000
- Backend API: http://localhost:5000/api

9. Folder Structure

The project follows a clean separation between frontend and backend codebases:

```
CampusEventHub_Team4/
├── backend/
│   ├── config/
│   ├── controllers/
│   ├── jobs/
│   ├── middleware/
│   ├── models/
│   ├── routes/
│   ├── tests/
│   ├── utils/
│   └── server.js
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── api/
│   │   ├── components/
│   │   ├── context/
│   │   ├── hooks/
│   │   ├── layouts/
│   │   ├── pages/
│   │   ├── services/
│   │   ├── styles/
│   │   └── utils/
│   ├── index.html
│   └── vite.config.js
```

10. Key Features

Feature 1: Event Discovery & Registration

Students can browse a searchable and filterable list of events, view full event details, and register via a registration modal. The interface provides an intuitive search bar, event cards, and a register button for streamlined participation.

- Backend endpoints: GET /api/events, POST /api/registrations
- Database: Events and registrations collections are updated in real time.
- User Action: Student browses events, selects preferred event, submits registration.

Feature 2: Admin Event Management

College administrators have access to a comprehensive dashboard featuring event creation and edit forms, registration management tables, approval toggles, and analytics visualizations.

- Backend endpoints: POST/PUT/DELETE /api/events, GET /api/registrations
- Database: Events and registrations collections updated on every admin action.
- User Action: Admin creates or edits events, approves or rejects student registrations.

Feature 3: Superadmin Platform Oversight

The superadmin role provides platform-wide oversight including approval of colleges and admins, analytics charts, and audit logs for monitoring system activity.

- Backend endpoints: /api/admin/colleges, /api/admin/logs
- Database: Colleges, users, and logs collections are updated accordingly.
- User Action: Approve or reject colleges and admins, monitor platform activity.

Feature 4: Secure Authentication & OTP

Authentication is handled using JWT for stateless session management, bcrypt for password hashing, and OTP-based email verification for account creation and password reset.

- Backend endpoints: /api/auth/login, /api/auth/register, /api/auth/verify-otp
- Database: Users collection updated with tokens and verification status.
- User Action: Login, register, OTP verification, and password reset.

11. Project Components

Frontend Components

App.jsx: Central routing configuration with role-based route protection.

ProtectedRoute.jsx: Handles role normalization, dashboard redirection, and debug logging.

DashboardLayout.jsx: Sidebar navigation with role-specific links and layout wrapper.

StudentDashboard.jsx: Student-facing dashboard for event browsing, registration tracking, and profile management.

CollegeAdminDashboard.jsx: Admin dashboard for event management, registration approvals, and analytics.

AdminDashboard.jsx: Superadmin dashboard for platform-level oversight and management.

ImageUpload.jsx: Robust image upload component with comprehensive error handling.

Backend Components

Controllers: Modular logic handlers for authentication, events, registrations, feedback, and notifications.

Models: Mongoose schemas for User, Event, Registration, Feedback, Notification, and related entities.

Routes: RESTful endpoint definitions for all application features.

Middleware: Authentication, input validation, and centralized error handling.

Jobs: Scheduled background tasks such as event reminders and cleanup routines.

12. API Components

All APIs follow REST principles and use JSON for communication. Authentication-protected routes require JWT tokens in request headers, ensuring secure access control. All endpoints are tested using Postman to ensure reliability and correctness.

User Registration

Route: /api/auth/register

Method: POST

Registers a new user and sends an OTP verification email.

Input: name, email, password, role

Output: { message: "Registration successful. Please verify your email." }

Status Codes: 201 (Created), 400 (Validation Error)

Flow: React form → API call → Controller validates input, hashes password, stores in MongoDB, sends OTP via SMTP.

User Login

Route: /api/auth/login

Method: POST

Authenticates an existing user and returns a signed JWT.

Input: email, password

Output: { token: "jwt_token_here", user: { id, role } }

Status Codes: 200 (Success), 401 (Invalid Credentials), 403 (Unverified Account)

Flow: React form → API call → Controller checks credentials, bcrypt compare, issues JWT on success.

Event Creation (Admin)

Route: /api/events

Method: POST

Allows an authenticated admin to create a new event.

Input: title, date, location, description

Output: { message: "Event created successfully.", eventId: "..." }

Status Codes: 201 (Created), 400 (Validation Error), 401 (Unauthorized)

Flow: Admin dashboard → API call → Controller validates and stores event in MongoDB.

Event Registration (Student)

Route: /api/registrations

Method: POST

Allows an authenticated student to register for a specific event.

Input: eventId

Output: { message: "Registration successful." }

Status Codes: 201 (Created), 400 (Bad Request), 401 (Unauthorized)

Flow: Event page → API call → Controller checks eligibility, stores registration record.

Notification Sending

Route: /api/notifications

Method: POST

Sends a targeted notification message to a specified user.

Input: userId, message

Output: { message: "Notification sent." }

Status Codes: 200 (Success), 400 (Bad Request)

13. Security Implementation

CampusEventHub follows industry-standard security practices across all layers of the application:

- Passwords are hashed using bcrypt before storing in MongoDB — plain-text credentials are never persisted.

- JWT tokens are used for stateless, scalable authentication across all protected routes.
- Protected routes validate tokens via middleware before granting access to any resource.
- Input validation at both the frontend and backend prevents invalid or malicious data from reaching the database.
- Sensitive operations are gated behind role-specific middleware, ensuring only authorized users can access or modify critical data.

14. System Flow

The end-to-end request lifecycle in CampusEventHub follows a clearly defined sequence:

1. User interacts with the React UI (e.g., clicks Register for Event).
2. React (via Axios) sends an HTTP request to the backend API.
3. Express route receives the request and forwards it to the appropriate controller.
4. Controller validates input and applies the required business logic.
5. Data is stored or retrieved from the appropriate MongoDB collection.
6. Backend sends a JSON response back to the client.
7. React dynamically updates the UI based on the response received.

This architecture ensures a clean separation of concerns between the UI layer, business logic layer, and data persistence layer.

15. Architecture

CampusEventHub follows a client-server architecture built on the MERN stack:

React (Client) ↔ Express.js REST API ↔ MongoDB

- JWT is used for authentication and session management.
- bcrypt is used for secure password hashing before storage.
- SMTP is used for email notifications, OTP delivery, and event reminders.

The frontend is fully decoupled from backend logic, enabling independent scaling and deployment of each tier. Horizontal scaling is supported by deploying multiple backend instances behind a load balancer and using a cloud-hosted MongoDB database such as MongoDB Atlas.

16. Error Handling & Troubleshooting

SMTP Failure: Verify SMTP credentials in the .env file. Check network connectivity and firewall rules for the SMTP port.

Port Conflict: Ensure ports 3000 (frontend) and 5000 (backend) are available. Change the port in configuration files if conflicts arise.

MongoDB Connection Failure: Verify the database URI in .env and confirm the MongoDB service is running.

API 401/404/500 Errors: Check JWT validity and expiry. Verify route existence in backend routes. Review server logs for stack traces.

JWT Expiry: Implement token refresh logic or redirect to re-login on the frontend when tokens expire.

File Upload Issues: Check file size and type limits. Verify Cloudinary or S3 configuration if cloud storage is used.

All errors are logged on the server for debugging and monitoring. Proper HTTP status codes are returned for all API responses to maintain consistency between frontend and backend.

17. Maintenance & Support

Database Backup: Use MongoDB Atlas automated backups or mongodump for local MongoDB instances.

Logging: Console logs are used in development. Integrate Winston or Morgan for structured production logging.

Monitoring: Use PM2 for process management, New Relic or Datadog for application performance monitoring, or configure built-in health check endpoints.

18. Version Control & Updates

Git Usage: All source code is managed using Git with a structured branching strategy.

Branching Strategy: main branch for production-ready code; feature/* branches for all new development.

Commit Format: Conventional Commits format is followed (e.g., feat: add event registration, fix: resolve JWT expiry bug).

19. Future Enhancements

The following features are planned for future versions of CampusEventHub:

- Integration with third-party calendar tools such as Google Calendar and Microsoft Outlook for seamless event scheduling.
- AI-powered event recommendations and intelligent filtering based on student preferences and academic interests.
- Advanced analytics dashboards with interactive charts and exportable reports for college admins and superadmins.
- Mobile application (React Native) to extend platform accessibility to smartphones.
- Real-time notifications using WebSockets for instant event updates and registration confirmations.

20. Glossary & FAQ

Glossary

Event: An activity, competition, or workshop organized by a college on the platform.

Registration: A student's formal sign-up for a specific event.

Admin: A college staff member responsible for managing that institution's events.

Superadmin: A platform-level administrator with authority over all colleges and admins.

OTP: One-Time Password — a time-sensitive code sent to a user's email for verification.

JWT: JSON Web Token — a stateless, signed token used for authentication.

Frequently Asked Questions

Q: How do I reset my password?

Use the Forgot Password link on the login page. An OTP will be sent to your registered email address.

Q: Can I register for multiple events?

Yes, students can register for any number of available events as long as they meet the eligibility criteria.

Q: How are events approved?

College admins create events which may require superadmin approval depending on platform configuration.

Q: Is my data secure?

Yes. Passwords are hashed with bcrypt, all API calls are protected with JWT, and sensitive configuration is stored in environment variables.

21. Conclusion

CampusEventHub successfully demonstrates a scalable, production-grade MERN stack architecture with real-world features including secure authentication, role-based access control, and comprehensive event lifecycle management. The system is designed to be modular, secure, and extensible, making it well-suited for further enhancements, institutional deployments, and integration with third-party services.

The platform addresses a genuine gap in campus activity management by unifying fragmented workflows into a single, accessible, and transparent system — empowering students, college administrators, and platform operators alike.

Developers

Uday (@udaycodespace) — Frontend Lead, Backend, Architecture, Design

Gayatri (@Gayatri-3168) — Backend

Documented by @udaycodespace

github.com/udaycodespace